

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ»

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНОЙ РОБОТОТЕХНИКИ

Кафедра Интеллектуальных систем теплофизики ИИР
Направление подготовки 09.03.03 Прикладная информатика
Направленность (профиль) Прикладной искусственный интеллект

ОТЧЕТ

о прохождении учебной практики (ознакомительной практики)

(указывается наименование практики)

Обучающегося Долгих Алексея Александровича группы № 25945 курса 1
(Ф.И.О. полностью)

Тема задания: Система семантического поиска по базе знаний компании

Место прохождения практики: Федеральное государственное автономное образовательное учреждение высшего образования «Новосибирский национальный исследовательский государственный университет», Мехатроника и робототехника. Искусственный интеллект. (Институт интеллектуальной робототехники НГУ), 630090, Новосибирская область, г. Новосибирск, ул. Пирогова, д. 1

(полное наименование организации и структурного подразделения, индекс, адрес)

Сроки прохождения практики: с 02.02.2026г. по 27.05.2026г.

Руководитель практики: Глушенко Андрей Валерьевич, ассистент кафедры интеллектуальных систем теплофизики ИИР (КафИСТИИР) /

(Ф.И.О. полностью, ученая степень, звание)

(подпись)

Оценка по итогам защиты отчета: _____

(неудовлетворительно, удовлетворительно, хорошо, отлично)

Отчет заслушан на заседании кафедры КафИСТИИР

(наименование кафедры)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. Проектирование архитектуры REST API.....	5
1.1. Технологический стек	5
1.2. Работа с документами	5
1.3. Работа с пользователями.....	5
1.4. Микросервис для обработки документов.....	6
2. Развёртывание и тестирование проекта	7
2.1. Обоснование использования контейнеров	7
2.2. Межсервисное взаимодействие.....	7
2.3. Автоматическое создание тестового набора данных	7
ЗАКЛЮЧЕНИЕ.....	8
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	10

ВВЕДЕНИЕ.

Актуальность проекта

Одной из ключевых проблем управления документами в компаниях является сложность навигации по сотням локальных файлов. Классический поиск по ключевым словам неэффективен, если сотрудник не знает точных формулировок или названий. Разработка интеллектуальной поисковой системы решает эту проблему за счет методов обработки естественного языка: алгоритм сопоставляет цель запроса пользователя с содержанием документов. Например, система будет способна связать разговорный запрос «хочу отдохнуть» с официальной инструкцией по оформлению отпуска.

Цель проекта

Целью нашего проекта является создание веб-приложения, которое позволит сотрудникам компании производить поиск по базе знаний, используя естественный язык, а не только ключевые слова.

Объект исследования

Архитектура и компоненты серверной части приложения системы семантического поиска документов.

Предмет исследования

Методы проектирования REST API [1], интеграции баз данных, контейнеризации приложений и организации межсервисного взаимодействия при разработке серверных решений.

Задачи

1. Спроектировать и разработать REST API для взаимодействия с клиентской частью приложения.
2. Настроить и развернуть базу данных (PostgreSQL [2]) для хранения информации о пользователях и документах.
3. Разработать микросервис [3] для автоматического извлечения и обработки текстового контента из файлов.

4. Настроить среду контейнеризации (Docker Compose [4]) для удобного управления и запуска всех компонентов системы (клиентской части, API, микросервиса языковой модели, обработчика документов и базы данных) единой командой.

5. Сформировать тестовый набор данных и проверить работоспособности системы.

Предполагаемые результаты

1. Развёрнутая локальная программная инфраструктура проекта на базе Docker Compose и PostgreSQL.

2. Спроектированная логика серверной части и реализованный интерфейс REST API, позволяющий пользователям загружать файлы и выполнять поиск документов.

3. Протестированное приложение с использованием сформированного набора документов.

1. Проектирование архитектуры REST API

1.1. Технологический стек

Разработка программного интерфейса (REST API) является ключевым этапом создания серверной части системы, так как именно этот компонент обеспечивает бизнес-логику приложения и связывает клиентскую часть с базой данных и микросервисами.

В качестве фреймворка для создания конечных точек доступа был выбран FastAPI [5]. Выбор обусловлен его высокой производительностью, встроенной поддержкой асинхронности [5] (что очень важно для микросервисной архитектуры) автоматической генерации документации [5] и удобной системой валидации данных [5].

Для организации хранения информации на сервере была выбрана реляционная СУБД PostgreSQL. Выбор обоснован поддержкой специализированных расширений, позволяющих сохранять векторные представления [6] и выполнять над ними математические операции для семантического поиска [6]. В рамках текущей архитектуры база данных содержит две таблицы: информация о документах и информация о пользователях.

1.2. Работа с документами

Для работы с документами был создан набор интерфейсных маршрутов [1], позволяющих загружать новые файлы на сервер, модифицировать их метаданные, а также удалять с сервера. Кроме того, разработаны конечные точки доступа для скачивания исходных документов и выполнения контекстного семантического поиска по всей базе знаний компании на основе вычисления косинусного сходства векторов [6].

1.3. Работа с пользователями

В рамках проектирования была предусмотрена возможность разграничения прав доступа на основе ролей. Данная модель даёт возможность разделить пользователей на обычных сотрудников (с правами только на поиск и чтение) и администраторов (с правами на модификацию базы знаний и управление учетными записями).

На текущем этапе разработки подсистема авторизации и проверки прав доступа была реализована в виде прототипа. На данный момент конечные точки доступа (создание,

изменение и удаление профилей пользователей) функционируют в открытом режиме без жесткой блокировки запросов [5].

1.4. Микросервис для обработки документов

Для того, чтобы модель смогла векторизовать документы, нужно извлечь текст из файлов. Для этих целей был разработан микросервис, который сможет конвертировать файлы в обычный текст с помощью библиотеки PyMuPdf [7].

Для тестов была добавлена возможность взаимодействовать только с PDF, но в перспективе можно добавить больше форматов или вообще заменить обработчик, что позволяет сделать микросервисная архитектура [3].

2. Развёртывание и тестирование проекта

2.1. Обоснование использования контейнеров

Микросервисная архитектура позволяет изолировать отдельные компоненты системы (серверная часть, обработчик документов, языковую модель, базу данных [8]) и производить их независимую модернизацию или замену без изменения общей структуры приложения.

Для управления созданными модулями и обеспечения их сквозной оркестрации был выбран инструмент Docker Compose [4], являющийся стандартом в современной индустрии разработки серверного ПО, а также использование Docker Compose решает ключевую задачу – кроссплатформенность.

2.2. Межсервисное взаимодействие

Сетевое взаимодействие между компонентами внутри Docker Compose изолировано от внешней среды. Сервисы обращаются друг к другу по протоколу HTTP, используя имена контейнеров из конфигурационного файла в качестве DNS-адресов [4].

Вся инфраструктура проекта описывается в едином конфигурационном файле, что позволяет развернуть, автоматически связать и запустить приложение одной командой [4].

2.3. Автоматическое создание тестового набора данных

Для проведения комплексного тестирования системы требовалось сформировать репрезентативный набор разнородных документов в формате PDF. В целях автоматизации было принято решение использовать API GigaChat [9] для генерации текста (документов, актов, инструкций) и Turpst [10] для компиляции текста в PDF.

Связующим звеном выступил скрипт на языке Python, который в автоматическом режиме отправлял запросы к ИИ-модели [9], обрабатывал текстовые ответы и с помощью утилиты Turpst формировал готовые файлы.

ЗАКЛЮЧЕНИЕ.

В ходе прохождения учебной ознакомительной практики были достигнуты все поставленные задачи. Была спроектированы и реализованы конечные точки доступа для управления жизненным циклом документов и учётных записей пользователей. Для хранения информации была развернута реляционная СУБД PostgreSQL с использованием расширения pgvector, которое позволило хранить векторное представление текста документов.

Спроектирован и внедрен микросервис предобработки входящих документов, работающий на базе программной библиотеки PyMuPDF. Данный модуль решил важную задачу автоматического извлечения чистого текстового контента из бинарных файлов для его последующей передачи в компоненты языковой модели.

Вся созданная комплексная инфраструктура была объединена и оркестрована с использованием инструмента Docker Compose. Развернутая конфигурация связала в единую изолированную виртуальную сеть центральное ядро серверной части, клиентский интерфейс, языковую-модель векторизации, сервис предобработки документов и базу данных. Настройка межсервисного взаимодействия позволила полностью автоматизировать процесс развертывания всего проекта одной командой.

На финальном этапе практики с использованием автоматизированных Python-скриптов, API GigaChat и утилиты компиляции Typst был сформирован набор тестовых документов. Созданный датасет позволил провести проверку работоспособности компонентов через встроенную интерактивную документацию FastAPI. Финальные результаты проверки показали стабильность разработанных маршрутов.

Данная работа позволила на практике освоить навыки написания микросервисной архитектуры и оркестрации распределительных систем. В процессе выполнения задания были успешно освоены навыки проектирования интерфейсов REST API, а также развертывания изолированных контейнерных сред в Docker Compose. Работа над модулем автоматической обработки документов и настройка сетевого взаимодействия между бэкендом и СУБД PostgreSQL позволили получить реальный опыт создания масштабируемых серверных

приложений. Поставленная цель практики достигнута в полном объеме, а созданный прототип обладает гибкой архитектурой, готовой к дальнейшему расширению функционала.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. REST, что же ты такое? Понятное введение в технологию для ИТ-аналитиков. [Электронный ресурс]. URL: <https://habr.com/ru/articles/590679/> (дата обращения: 02.03.2026).
2. Что такое PostgreSQL. [Электронный ресурс]. URL: <https://metanit.com/sql/postgresql/1.1.php> (дата обращения: 10.04.2026).
3. Просто о микросервисах. [Электронный ресурс]. URL: <https://habr.com/ru/companies/raiffeisenbank/articles/346380/> (дата обращения: 04.03.2026).
4. Docker для начинающих: что это такое и как пользоваться. [Электронный ресурс]. URL: <https://habr.com/ru/companies/netologyru/articles/967546/> (дата обращения: 04.04.2026).
5. FastAPI — официальная документация. [Электронный ресурс]. — URL: <https://fastapi.tiangolo.com/ru/> (дата обращения: 13.03.2026).
6. Векторный поиск внутри PostgreSQL: что умеет и где может пригодиться pgvector. [Электронный ресурс]. URL: <https://habr.com/ru/companies/selectel/articles/920824/> (дата обращения: 06.04.2026).
7. PyMuPDF: A high-performance Python library for data extraction, analysis, conversion & manipulation of PDF (and other) documents. [Электронный ресурс]. URL: <https://pypi.org/project/PyMuPDF/> (дата обращения: 15.05.2026).
8. Запускаем PostgreSQL в Docker: от простого к сложному. [Электронный ресурс]. URL: <https://habr.com/ru/articles/578744/> (дата обращения: 15.04.2026).
9. GigaChat: Быстрый старт для физических лиц. [Электронный ресурс]. URL: <https://developers.sber.ru/docs/ru/gigachat/individuals-quickstart> (дата обращения: 15.05.2026)
10. Typst — официальная документация. [Электронный ресурс]. URL: <https://typst.app/docs/tutorial/formatting/> (дата обращения: 15.05.2026).